CSCI 235 — Class 16 Review

Pruning and Early Termination: Your Questions Answered

Hunter College, Fall 2026

Your lecture slips from Class 16 asked good questions, and several of them were the same question. This handout answers them in order: `rem`, the mysterious `1e-9`, duplicate letters, more examples of pruning and early termination, where the big numbers (1023, 244, 12, 4,141, 139, ...) come from, the key ideas with related problems, and extra practice. Every call count below was checked by running the code (see `class16_review_code.cpp`).

Three sentences to hold onto. (1) `rem` is "how much do I still need?" **(2)** `1e-9` means $1 \times 10^{-9} = 0.000000001$, our "close enough" distance for decimals. **(3)** Early termination says "stop everything, I'm done"; pruning says "skip this branch, it cannot help."

1 What is `rem`, and what is its point?

"What is rem? What is the point of rem?"

Analogy: a gift card. You hold a \$6 gift card and want to spend it *exactly*. The shop sells items priced $4, 1, 2, 3$. You walk down the shelf one item at a time and decide: *take it* or *skip it*. After each decision you ask yourself, "How much of the card is still unspent?" That number is `rem`, short for *remaining*.

Variable	Question it answers	Gift-card meaning
`k`	Which item am I looking at?	position on the shelf
`rem`	How much do I still need?	unspent money on the card
`chosen`	What is already in my basket?	items taken so far

The target never changes; `rem` does. Taking `v[k]` uses up part of the card, so the next call gets `rem - v[k]`. Skipping changes nothing, so the next call gets `rem` unchanged.

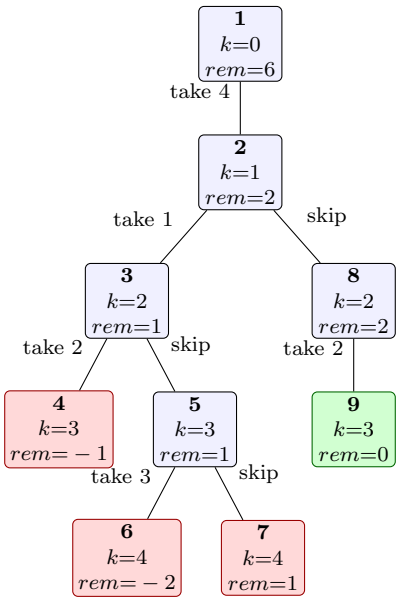
```
bool subsetSum(const vector<int>& v, int k, int rem, vector<int>& chosen) {
    if (rem == 0) return true;              // card spent exactly: success
    if (k == (int)v.size()) return false;  // no items left
    if (rem < 0) return false;              // overspent: prune
    chosen.push_back(v[k]);                 // take v[k]
    if (subsetSum(v, k + 1, rem - v[k], chosen)) return true;
    chosen.pop_back();                      // undo
    return subsetSum(v, k + 1, rem, chosen); // skip v[k]
}
```

Why keep `rem` instead of adding up `chosen` each time? Because `rem` *is* the answer to the only question the search cares about, and it travels down the recursion for free. Each call tests `rem == 0` and `rem < 0` in one step. It also makes pruning possible: "I have already overspent" is just `rem < 0`.

A complete trace: $v = \{4, 1, 2, 3\}$, target 6

Follow the calls in the order they happen (take is tried before skip). The search makes **9 calls** and returns `chosen = {4, 2}`.

Call	`k`	`rem`	`chosen` on entry	What happens
1	0	6	{}	take 4
2	1	2	{4}	take 1
3	2	1	{4,1}	take 2
4	3	−1	{4,1,2}	overspent: `rem < 0`, return false
5	3	1	{4,1}	back up, skip 2, then take 3
6	4	−2	{4,1,3}	no items left, return false
7	4	1	{4,1}	skip 3: no items left, return false
8	2	2	{4}	back up, skip 1, then take 2
9	3	0	{4,2}	`rem == 0`: success



Blue: ordinary call. Red: returns `false`. Green: success. Numbers are call order.

Notice: once call 9 returns `true`, every call above it returns `true` immediately, and nothing else is tried. That is *early termination*. Without the `rem < 0` test, the same search takes **11** calls: call 4 would keep going instead of stopping.

Why call 4 is pruned but call 6 is not "pruned". Call 6 has `rem` $= -2$ too, but its `k` $= 4 = n$, so no elements are left and it returns `false` on its own: there is nothing more to expand, with or without the `rem < 0` test. Call 4 has `k` $= 3$, so one element (the 3) is still unprocessed; without the test it would make two more calls (take 3, skip 3). That is exactly the 2 calls saved: $9 + 2 = 11$.

Common trap. The test `rem < 0` is safe only when every remaining number is *nonnegative*. If negative numbers are allowed, taking -3 later could bring `rem` back up to 0 (Section 4).

2 What does `1e-9` mean?

"What does 1e-9 mean?"

Reading it. `1e-9` is scientific notation: 1×10^{-9}, which is 0.000000001 (a billionth). The `e` stands for "times ten to the power of". So `2.5e3` is 2500 and `1e-9` is a decimal point followed by eight zeros and a one. It is *not* "1 to the power 9" and not 10^9.

Why does Make 24 need it? Computers store decimals like `0.1` in binary, which cannot represent them exactly. Try this:

```
cout << (0.1 + 0.2 == 0.3);              // prints 0 (false!)
cout << (abs(0.1 + 0.2 - 0.3) < 1e-9);   // prints 1 (true): gap is about 5.6e-17
```

Mathematically $0.1 + 0.2 = 0.3$, but the stored value of $0.1 + 0.2$ is 0.30000000000000004. Two decimals that "should" be equal can differ by a speck.

Analogy: a tape measure. If you measure a board twice and get 24.0000 cm and 24.0001 cm, you say "same length". Nobody demands the atoms line up. `1e-9` is the width of the "same length" window for our program: if two numbers differ by less than a billionth, we treat them as equal.

Where it appears in the Make 24 solver.

Code	Meaning
`std::abs(nums[0] - 24) < 1e-9`	"Is the last number close enough to 24?"
`std::abs(b) > 1e-9`	"Is b far enough from 0 to divide by it?"

The first replaces `nums[0] == 24`, which can fail on a correct answer like $8 \div (3 - 8 \div 3)$. The second avoids dividing by a number that is zero *or so close to zero* that the quotient would be huge and unreliable. The tiny value might be a rounding leftover (for example, $0.1 + 0.2 - 0.3$ is about 5.6×10^{-17}, not exactly 0) or genuinely small; either way we skip the division. (Exact cases like $1.0 - 1.0$ give exactly 0, which the guard also catches.)

Is 10^{-9} a law of nature? No. It is a *numerical policy* that works for small inputs like card values. Integers (like the subset-sum problem) need no tolerance at all: $2 + 4 == 6$ is always exact.

3 Duplicate letters: `printAll` versus `printUnique`

"Can we go over duplicate letters please?"

The problem. `printAll("AAB")` makes every arrangement by swapping, and it treats the two `A`s as different people. It prints $3! = 6$ lines: `AAB, AAB, ABA, ABA, BAA, BAA`. Only **3** are different. We want each distinct arrangement once.

Analogy: filling seat k. The letters still unplaced are the candidates for the seat. If two candidates are twin `A`s, putting either twin in the seat gives the same picture. So at each seat, try each *letter value* once.

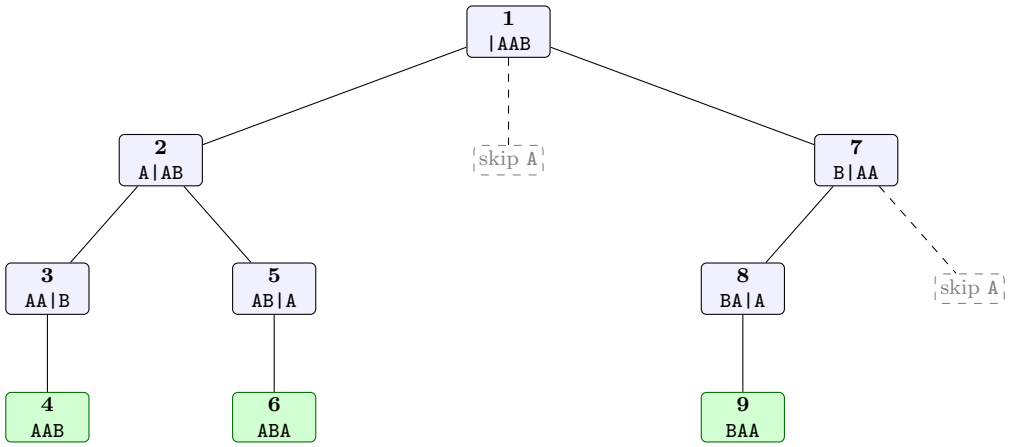
```cpp
void printUnique(string s, int k) {
    if (k == (int)s.size()) { cout << s << "\n"; return; }
    set<char> tried;                      // LOCAL: new for every call
    for (int i = k; i < (int)s.size(); ++i) {
        if (tried.count(s[i])) continue; // already tried this letter at seat k
        tried.insert(s[i]);
        swap(s[k], s[i]);
        printUnique(s, k + 1);
        swap(s[k], s[i]);                // undo
    }
}
```

Two facts about `set` (used in `tried` and `seen`)

1. A set ignores insertion order, and each element appears at most once. A `vector` remembers order, so `{1, 2}` and `{2, 1}` are *different* vectors. In a `set`, `{1, 2}` and `{2, 1}` are the *same* set: all that matters is which elements are inside. *Math versus C++:* in both, **an element appears at most once** (a repeated insert is ignored). The only difference is in how the set is stored: a mathematical set has no order at all, while a `std::set` keeps its elements in **sorted order** internally. So inserting 2 then 1 gives the same stored set as inserting 1 then 2, and looping over it always visits $1, 2$ in increasing order. (This is also why the Make 24 key is sorted: sorting makes two vectors with the same numbers in different orders identical.)

2. `count` and truth. `tried.count(x)` returns how many times `x` appears in the set. A `set` keeps each element at most once, so the answer is `0` (not there) or `1` (there). In C++, `0` counts as `false` and any nonzero number counts as `true`, so `if (tried.count(s[i]))` means "if `s[i]` is already in `tried`". (A `multiset` can return 2, 3, ..., but the `if` still works the same way.)

In the picture, `prefix|rest` shows the letters fixed so far (left of the bar) and the letters still to place. Dashed boxes are choices skipped because that letter was already tried at this seat.



`printUnique("AAB",0)`: 9 calls, 3 lines. `printAll` makes 16 calls and prints 6 lines.

Why must `tried` be local? It answers "which letters have I tried *at this seat of this call*?" Another call, at another seat or under another prefix, is a different question with a different answer.

A global set would wrongly forbid letters in unrelated places. (A letter may appear in many seats of the output; that is fine.)

Word	`printAll` calls	`printAll` lines	`printUnique` calls	`printUnique` lines
`ABC`	16	6	16	6
`AAB`	16	6	9	3
`AAA`	16	6	4	1
`AABB`	65	24	19	6

Distinct arrangements: $\frac{n!}{(\text{repeat counts})!}$, e.g. $\frac{4!}{2!\,2!} = 6$ for `AABB`. When all letters differ (`ABC`) there is nothing to skip, so `printUnique` saves nothing, which is exactly right.

4 More uses of pruning and early termination

"Can you go over more special examples and use cases?"

First, the vocabulary again.

	Early termination	Pruning
Idea	Stop the *whole* search after a success.	Skip *one branch* that cannot help.
Needs	You want *one* answer (does it exist? find one).	A reason the branch is hopeless or redundant.
Helps when	A solution exists, found early.	Often, even with no solution.
Does nothing when	You must count or print *every* solution.	The test never fires.

Safety question for every prune: "Could a solution live inside the branch I am about to skip?" If you cannot say *no*, do not prune.

Use case A: subset sum with nonnegative numbers

Two prunes: `rem < 0` (overspent) and `rem > suffix[k]` (even taking every remaining item cannot reach the target; `suffix[k]` is the sum of `v[k]` through `v[n-1]`). Example: $v = \{1, 2, 3, 4\}$, target 11. The numbers add to 10, so there is no solution.

Version	Calls
Early termination only	31
+ `rem < 0`	31
+ `rem > suffix[k]`	1

Here `rem < 0` never fires (nothing overshoots before the numbers run out), yet the bound kills the whole tree at the root. Different prunes help in different situations.

Use case B: when a prune is *unsafe*

Let $v = \{5, -3\}$ and target 2. Taking 5 gives `rem = -3`. A `rem < 0` prune would stop here and report "no solution". But the next number -3 brings `rem` back to 0: the answer `{5, -3}` exists. With the unsafe prune you lose a real solution; without it the search makes 3 calls and finds it.

Use case C: pruning that barely helps

$v = \{2, 4, 6\}$, target 7 (take before skip). All numbers are even and the target is odd, so there is no solution, but our tests do not know about parity. Here is exactly which tests each version uses. All three use the same call order: a call first tests `rem == 0`, then `k == n`, then (if enabled) the pruning tests, then takes and skips. Every function entry counts as one call, including the root and the calls that return at once.

Version	Tests at function entry	Calls
A	`rem == 0`, `k == n` (no pruning)	15
B	A + `rem < 0`	15
C	B + `rem > suffix[k]` (with `suffix = {12, 10, 6, 0}`)	13

Why B saves nothing: the sums only go negative on the very last number: $2 + 4 + 6 = 12$ gives `rem` $= -5$, and similar cases give -3 and -1, always at `k` $= 3$, where `k == n` already stops the call. So `rem < 0` never fires early. **Why C saves 2:** after skipping 2 and 4, the call has `rem` $= 7 >$ `suffix[2]` $= 6$, so it is rejected, and its two children disappear: $15 - 2 = 13$.

(*A different design* tests *before* making the call, for example "only take `v[k]` if `rem - v[k] >= 0`". That version never enters the three negative-`rem` calls and makes 12 calls, or 10 with the bound tested the same way. Counts depend on where the test is placed, so always state it.)

A parity prune would help more, but only if you can *prove* it is safe.

Use case D: 4-Queens, check early or check late

Place one queen per row. **No pruning:** try every column in every row and check conflicts only when the board is full. **Pruning:** before placing a queen, check it against earlier queens and skip conflicting columns. Either version can also use early termination (stop at the first full board).

	Find all solutions	Stop at first solution
No pruning (check at the end)	341 calls	155 calls
Pruning (check as you place)	17 calls	9 calls

Moral: pruning shrinks the tree; early termination cuts the search short. Together they save the most. Early termination alone does nothing if you must *find all* solutions.

Use case E: Make 24, duplicate states

In Make 24, {2,1,1} and {1,2,1} are the same state, so we sort a key and keep a `seen` set. For $\{1, 1, 1, 1\}$ (no solution): 9,841 calls with ordered pairs, 4,141 with unordered pairs and six results,

and 139 with `seen` (reference counts for the lecture implementations). Early termination cannot fire when no solution exists, but duplicate-state pruning still helps a lot.

Careful: `return false` for a key already in `seen` is justified only if (1) equal keys really mean equivalent states (exact keys) and (2) a success returns `true` all the way out. Rounded keys can collide, so (1) is not guaranteed.

If the problem says...	Think...
"Does a solution exist?" / "Find one"	early termination
"Print / count all solutions"	no early termination; prune only
"The partial answer is already invalid"	infeasibility pruning
"I have been in this exact situation before"	duplicate-state pruning (check exactness!)
"Negative numbers / unusual rules"	re-check every prune

5 Where do the numbers come from?

"How do numbers like 4,141 and 9,841 come into being?"

Every number here is one of two things: a **product** of choices ("how many ways to pick a first move, then a second move...") or a **count of calls**, meaning nodes in the call tree, including the root and the calls that return at once. We rebuild the Make 24 numbers first, since 244 and 12 were explained in class (a short recap is at the end).

The big picture

The goal of this course is to understand the *algorithms*. The numbers matter only because they show **how fast the work grows** and **how much pruning saves**. You do *not* need to memorize or rederive every count. Here is what to take away:

1. The number of choices **multiplies** from layer to layer, so the full tree is huge: $36 \times 18 \times 6 = 3{,}888$ sequences for 4 numbers, and $60 \times 36 \times 18 \times 6 = 233{,}280$ for 5.
2. Pruning and duplicate detection cut the real work a lot: about 4,000 calls shrink to 139 for $\{1, 1, 1, 1\}$ because there are only 13 different states.
3. Exact counts such as 4,141, 9,841, 1,401 and 749 depend on the implementation (guards, operation order). They are *reference numbers*, not facts to memorize.

Core (read this): Steps 0, 1, 2 and the first paragraph of Step 5. **Optional** (skip unless curious): Steps 3, 4, 6 and the level-by-level detail in Step 5.

Step 0 (core): what is a "move", and what is the "first move"?

A **move** turns a state with n numbers into a state with $n - 1$ numbers. It has two parts:

(1) Choose 2 of the n numbers (a pair; order does not matter).
(2) Choose 1 of the 6 operations for that pair ($a{+}b$, $a{\times}b$, $a{-}b$, $b{-}a$, $a{\div}b$, $b{\div}a$). The two chosen numbers are replaced by that one result.

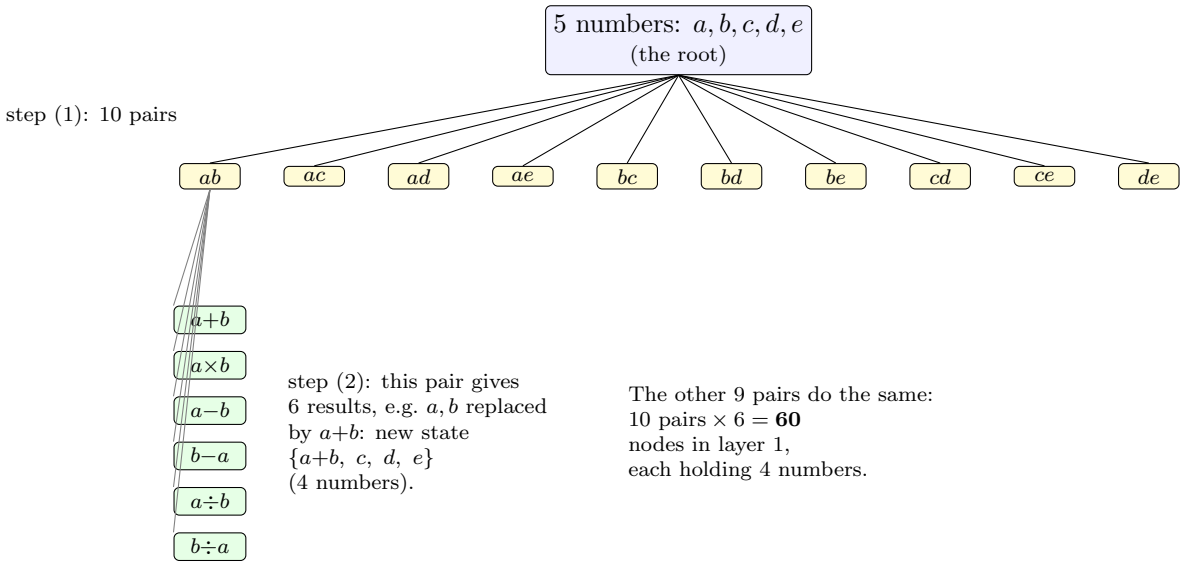
The **first move** is the very first one, going from the 5 starting numbers to 4 numbers. It is the first layer of the call tree: the root has one child for every (pair, operation) choice.

Count the pairs. Take numbers a, b, c, d, e. Number a can pair with 4 others, b with 4 others, and so on, giving $5 \times 4 = 20$ *ordered* pairs. But (a, b) and (b, a) are the same pair, so we divide by 2:

$$\frac{5 \times 4}{2} = 10 \text{ pairs:} \quad ab,\ ac,\ ad,\ ae,\ bc,\ bd,\ be,\ cd,\ ce,\ de.$$

Count the operations. Each of the 10 pairs gives 6 results, so each pair spreads out into 6 children:

$$\text{nodes in layer 1} = \frac{5 \times 4}{2} \times 6 = 10 \times 6 = \mathbf{60}.$$



Every later layer works the same way, only with fewer numbers:

Move	Numbers before → after	Pairs	Nodes spread out from *one* state
1st (the "first move")	$5 \to 4$	$\frac{5 \cdot 4}{2} = 10$	$10 \times 6 = \mathbf{60}$
2nd	$4 \to 3$	$\frac{4 \cdot 3}{2} = 6$	$6 \times 6 = 36$
3rd	$3 \to 2$	$\frac{3 \cdot 2}{2} = 3$	$3 \times 6 = 18$
4th	$2 \to 1$	$\frac{2 \cdot 1}{2} = 1$	$1 \times 6 = 6$

A complete game from 5 numbers chooses one option at every move, so the number of complete sequences is the product $60 \times 36 \times 18 \times 6 = 233{,}280$. The same idea with 4 numbers starts at the second row of the table: $36 \times 18 \times 6 = 3{,}888$, which is the number in Step 1 below.

Step 1: the product rule for 4 numbers (36, 18, 6, 3,888, and 48, 24, 8, 9,216)

A move picks a pair and one result for it. With four numbers on the table, unordered pairs and 6 results per pair:

$$\text{first move: } \tbinom{4}{2} \times 6 = 36, \qquad \text{second: } \tbinom{3}{2} \times 6 = 18, \qquad \text{third: } \tbinom{2}{2} \times 6 = 6.$$

Any first move can be followed by any second move, so we *multiply*: $36 \times 18 \times 6 = \mathbf{3{,}888}$ complete sequences. For the ordered version (each ordered pair, only 4 results): $(4 \cdot 3) \times 4 = 48$, then $(3 \cdot 2) \times 4 = 24$, then $(2 \cdot 1) \times 4 = 8$, so $48 \times 24 \times 8 = \mathbf{9{,}216}$. The "$4 \cdot 3$" is "4 numbers, each with 3 friends".

Step 2: sequences are not calls (core)

3,888 counts only the *last* level. A call happens at every level, so add them all:

$$\text{ideal calls} = \underbrace{1}_{\text{root}} + \underbrace{36}_{\text{first moves}} + \underbrace{36 \cdot 18}_{648} + \underbrace{36 \cdot 18 \cdot 6}_{3888} = 4573 \quad \text{(unordered)},$$

$$1 + 48 + 48 \cdot 24 + 48 \cdot 24 \cdot 8 = 1 + 48 + 1152 + 9216 = 10{,}417 \quad \text{(ordered)}.$$

The lecture reports **4,141** and **9,841**, which are smaller. The difference comes from one rule in the code: *a division is skipped when the divisor is zero.* For $\{1, 1, 1, 1\}$, $1 - 1 = 0$ creates a zero, and every skipped division removes a call *and* the whole subtree that would have hung below it.

Big picture: the guard trims the tree only slightly (4573 → 4141, about 9%). The exact totals below are optional; what matters is that the tree has thousands of calls.

Step 3: exact derivation of 4,141 (unordered pairs, 6 results) [OPTIONAL]

Optional: extra detail for the curious

Level with 3 numbers: 36 calls. Each first move gives one of three states:

State after move 1	Number of moves producing it	Which results make it
$\{1, 1, 2\}$	6	$1 + 1 = 2$
$\{1, 1, 1\}$	18	1×1, $1 \div 1$, $1 \div 1$ (6 pairs each)
$\{1, 1, 0\}$	12	$1 - 1$, $1 - 1$ (6 pairs each)
Total	**36**	

Level with 2 numbers: 624 calls. Each of those states has 3 pairs, each pair gives 6 results, so $3 \times 6 = 18$ children, except $\{1, 1, 0\}$ loses two moves: the pairs $(1, 0)$ and $(1, 0)$ each skip $1 \div 0$, so it has $18 - 2 = 16$ children.

$$6 \times 18 + 18 \times 18 + 12 \times 16 = 108 + 324 + 192 = \mathbf{624}.$$

(The ideal was $36 \times 18 = 648$: the shortfall is $12 \times 2 = 24$.)

Level with 1 number: 3,480 calls. Each two-number state has exactly one pair, so it makes up to 6 calls, minus any $\div 0$ skipped. A state with no zero makes 6; with one zero it makes 5 (one division skipped); with two zeros $\{0, 0\}$ it makes 4 (both divisions skipped). Counting the 624 states by how many zeros they contain:

Zeros in the 2-number state	0	1	2
How many states (from the program)	384	216	24
Calls each makes	6	5	4
Calls	$384 \times 6 = 2304$	$216 \times 5 = 1080$	$24 \times 4 = 96$

$$2304 + 1080 + 96 = \mathbf{3480}.$$

Total: $1 + 36 + 624 + 3480 = \mathbf{4{,}141}$.

Step 4: exact derivation of 9,841 (ordered pairs, 4 results) [OPTIONAL]

Optional: extra detail for the curious

3 numbers: 48 calls. There are 12 ordered pairs and four operations: $1 + 1 \to \{1, 1, 2\}$ (12 moves), $1 - 1 \to \{1, 1, 0\}$ (12 moves), 1×1 and $1 \div 1 \to \{1, 1, 1\}$ (24 moves). Total $12 + 12 + 24 = 48$.

2 numbers: 1,128 calls. In a 3-number state there are $3 \cdot 2 = 6$ ordered pairs and 4 operations: $6 \times 4 = 24$ children. $\{1, 1, 0\}$ has two ordered pairs of the form $(1, 0)$, whose $1 \div 0$ is skipped, so it makes $24 - 2 = 22$.

$$12 \times 24 + 24 \times 24 + 12 \times 22 = 288 + 576 + 264 = \mathbf{1128}.$$

1 number: 8,664 calls. A two-number state has 2 ordered pairs, so $2 \times 4 = 8$ results, minus skipped $x \div 0$: one zero skips 1 (7 children), two zeros skip 2 (6 children).

Zeros in the 2-number state	0	1	2
How many states (from the program)	792	312	24
Calls each makes	8	7	6
Calls	$792 \times 8 = 6336$	$312 \times 7 = 2184$	$24 \times 6 = 144$

$$6336 + 2184 + 144 = \mathbf{8664}.$$

Total: $1 + 48 + 1128 + 8664 = \mathbf{9{,}841}$.

	4 numbers	3 numbers	2 numbers	1 number	Total
Ordered: ideal	1	48	1,152	9,216	10,417
Ordered: actual	1	48	1,128	8,664	**9,841**
Unordered: ideal	1	36	648	3,888	4,573
Unordered: actual	1	36	624	3,480	**4,141**

The zero counts (384, 216, 24 and 792, 312, 24) come from listing every state; the program prints the totals per level, and you can confirm them with `class16_review_code.cpp`. The exact totals depend on the guard ("skip ÷0") and on how zeros are handled, which is why these are *reference counts* for the lecture code.

Step 5 (core idea, optional details): why 139 is so much smaller

In $\{1, 1, 1, 1\}$ the 36 first moves lead to only **3 different states**, and the whole search visits only **13 different states** $(1 + 3 + 9)$. Without `seen` we re-explore the same state over and over; with `seen`, a repeated state costs one cheap call instead of a whole subtree. That is the idea; the next box counts it exactly.

Optional: extra detail for the curious

- **3 numbers: 36 calls.** The 36 first moves still make 36 calls, but they land on only **3 different states**: $\{1,1,2\}$, $\{1,1,1\}$, $\{1,1,0\}$. Only the first call to each is expanded; the other 33 find their key in `seen` and return `false` at once.
- **2 numbers: 52 calls.** Only the 3 expanded states make children: $18+18+16=$ **52**.
- **1 number: 50 calls.** Those 52 calls reach only **9 different two-number states**. Each expands once into at most 6 results: $9\times 6=54$ minus 4 skipped divisions by zero = **50**.

$$1+36+52+50=\textbf{139}.$$

Distinct states expanded: $1+3+9=13$.

Step 6: {3,3,8,8}, 1,401 and 749 [OPTIONAL]

Optional: extra detail for the curious

These are *not* products. The search stops when it finds $8\div(3-8\div 3)=24$, so each total is the number of calls made *before* success. The program prints the levels: $1+12+203+1185=1401$ without `seen`, and $1+12+169+567=749$ with it. They depend on the operation order $a+b$, $a*b$, $a-b$, $b-a$, a/b, b/a; another order would find the answer sooner or later and give different counts.

Recap: 1023, 244, 12 (subset sum, explained in class)

For $v=\{9,8,\ldots,1\}$ and target 20: $1023=1+2+4+\cdots+512=2^{10}-1$ (no savings); with early termination and `rem < 0` the search makes **12** calls, and without the `rem < 0` test the four overshooting calls (4, 6, 8, 10) each grow a wasted subtree: $12+126+62+30+14=$ **244**.

6 Key ideas and related problems

"The subject is dense. What are the main ideas, and where else do they appear?"

The big picture

Everything in this lecture is one idea: **try a choice, explore, undo it, and avoid wasted work.** The wasted-work tricks come in three flavors: stop everything once you are done, skip branches that cannot work, and skip branches you have already seen.

The five key ideas

Idea	Everyday picture	In our code
Backtracking (choose, explore, undo)	Walking a maze with chalk: mark a turn, go down it, rub it out if it fails.	`push_back`, recursive call, `pop_back`
Early termination	You found the exit, so you stop looking; you do not check the remaining corridors.	`if (solve(...)) return true;`
Infeasibility pruning	A "dead end" sign: no point walking down.	`rem < 0`, `rem > suffix[k]`, a queen that is attacked
Duplicate-state pruning	"I have been in this room before"; no need to search it again.	`seen` set, `tried` set
Safety check	Before posting a dead-end sign, be sure there is really no exit behind it.	"Could a solution live in this branch?"

One template that fits them all

Almost every problem in this lecture, and many interview problems, is this skeleton with the blanks filled in:

```
bool search(State& s) {
    if (isGoal(s))      return true;    // success
    if (isHopeless(s))  return false;   // infeasibility pruning (must be safe!)
    if (alreadySeen(s)) return false;   // duplicate-state pruning (keys must be exact)
    for (Choice c : choices(s)) {
        apply(s, c);                     // choose
        if (search(s)) return true;      // explore + early termination
        undo(s, c);                      // undo
    }
    return false;
}
```

Why is it fine to `return true` without undoing? After a success we skip `undo(s, c)`. That is safe only when the caller does not need the shared state restored. In subset sum, `chosen` *is* the answer we want to keep. In our Make 24 solver, `rest` is a local copy, so nobody else sees it. If the state is shared and the caller will keep searching or reuse it, you must undo before returning.

Problem	Goal	Hopeless	Choices
Subset sum	`rem == 0`	`rem < 0`, `rem > suffix[k]`	take or skip `v[k]`
`printUnique`	`k == n`	(none)	each *new* letter at seat `k`
N-Queens	all rows filled	queen attacked	a column in this row
Make 24	one number, equal to 24	(none; use `seen`)	pair × operation

Two related problems

R1. Partition into two equal halves. Given $\{1, 5, 11, 5\}$, can you split the numbers into two groups with the same sum? *Hint:* the total is 22, so each group needs sum 11; this is subset sum with target total/2. If the total is odd you can answer "no" immediately with no search (a free prune). For this input the search finds $\{11\}$, so the answer is yes.

R2. Count the letter sequences. How many different nonempty sequences can you make from the tiles `A, A, B` (each tile used at most once)? *Hint:* same "try each new letter once per seat" idea as `printUnique`, but count every prefix too. The answer is 8: `A, B, AA, AB, BA, AAB, ABA, BAA`.

Related LeetCode problems

These use the same ideas (names and numbers as on LeetCode; try them in C++).

Topic in this handout	Problems	Why they match
Take/skip choices, `rem`	78 Subsets; 39 Combination Sum; 216 Combination Sum III; 416 Partition Equal Subset Sum; 494 Target Sum	`rem` is the amount still needed; prune when it is negative
Duplicates (`tried` set)	47 Permutations II; 90 Subsets II; 40 Combination Sum II; 1079 Letter Tile Possibilities	skip a value already tried at this position
Prune by a constraint	51 N-Queens; 52 N-Queens II; 37 Sudoku Solver; 22 Generate Parentheses; 79 Word Search	check validity *as you place*, not at the end
Same family as Make 24	679 24 Game	the Project 3 problem with a tolerance for division
Warm-up (no pruning)	46 Permutations; 77 Combinations	the plain choose–explore–undo pattern

The subset-style problems share the take/skip recursion, but the `rem < 0` prune does **not** apply to all of them. It fits when there is a target and the numbers are nonnegative (39, 40, 216, 416). Subsets (78, 90) have no target, so there is no `rem`. Target Sum (494) uses $+/-$ signs, so you would compare against the sum of the remaining numbers instead. Check the problem's rules before pruning.

Suggested order: 78, 46, 47, 39, 51, 679.

7 Practice problems

Try each before looking at the solutions in Section 9.

P1. Trace. For $v = \{6, 1, 2\}$ and target 3 (take before skip), write each call as (k, rem) and state how many calls are made (a) with only early termination and (b) with early termination and the `rem < 0` test. What is `chosen`?

P2. Bound. For $v = \{1, 2, 3, 4\}$ and target 11, how many calls with early termination only, and how many when you also use `rem > suffix[k]`? Explain in one sentence why the second answer is so small.

P3. Safe or unsafe? For each, say whether `rem < 0` is a safe prune: (a) $v = \{3, 1, 4\}$, target 5; (b) $v = \{5, -3\}$, target 2; (c) $v = \{0, 2, 7\}$, target 9.

P4. Tolerance. (a) Write `1e-9` as a decimal. (b) Which is true in C++: `0.1 + 0.2 == 0.3` or `abs(0.1 + 0.2 - 0.3) < 1e-9`? (c) Why is no tolerance needed when adding `int`s?

P5. Duplicate letters. For `ABB`: how many distinct arrangements are there, and how many calls does `printUnique("ABB", 0)` make? (Hint: same shape as `AAB`.)

P6. Local or global? A student declares `set<char> tried;` *outside* `printUnique`. Explain what goes wrong for `"AB"`.

P7. Make 24 counting. With 5 numbers, how many first moves are there if a move picks an unordered pair and one of 6 results? How many numbers remain after one move? Explain where each factor of your product comes from.

P8. Which technique? Say whether early termination, pruning, both, or neither helps: (a) printing all subsets that sum to 6; (b) deciding whether any subset sums to 6; (c) 4-Queens, find all solutions.

P9. Derive it (challenge). (a) For $v = \{5, 4, 3, 2, 1\}$ with no pruning and no early termination, how many calls? (b) With 3 numbers on the table (unordered pairs, 6 results), how many complete move sequences are there?

8 Practice code

Complete these on your own, then compare with `class16_review_code.cpp`.

C1. Add a global counter `calls` to `subsetSum` and print it for $v = \{4, 1, 2, 3\}$, target 6. Confirm 11 without the `rem < 0` test and 9 with it.

C2. Write `countSubsets(v, k, rem)` that returns the *number* of subsets summing to the target. Which technique no longer applies? Which still does?

C3. Write `bool nearlyEqual(double x, double y)` and test it on $0.1 + 0.2$ versus 0.3.

C4. Modify `printUnique` to *count* distinct arrangements instead of printing them. Check that `AABB` gives 6.

C5. N-Queens for $n = 5$: add a call counter and compare "check at the end" with "check as you place".

9 Solutions

P1. (a) **11** calls. (b) **5** calls: $(0, 3)$; take 6 gives $(1, -3)$, pruned; skip 6 gives $(1, 3)$; take 1 gives $(2, 2)$; take 2 gives $(3, 0)$, success. `chosen = {1, 2}`.

P2. 31 calls, then **1**. The numbers add to $10 < 11$, so even taking everything cannot reach 11, and the root call is rejected immediately.

P3. (a) safe (all nonnegative); (b) unsafe (a negative number can raise `rem` again); (c) safe (zero and positives are nonnegative).

P4. (a) 0.000000001. (b) The tolerance test; the exact comparison is false. (c) Integer arithmetic is exact, so equal values compare equal.

P5. $3!/2! = 3$ arrangements; **9** calls.

P6. A global `tried` remembers letters from earlier calls. For `"AB"`: seat 0 tries `A`, and the call at seat 1 inserts `B` and prints `AB`. Back at seat 0, the loop reaches `B`, finds it already in the shared set, and skips it, so `BA` is never printed.

P7. $\binom{5}{2} \times 6 = 10 \times 6 = 60$ first moves; 4 numbers remain.

P8. (a) pruning only (all answers are needed, so no early stop). (b) both. (c) pruning only.

P9. (a) 5 numbers give 6 levels: $2^6 - 1 = 63$ calls. (b) $\binom{3}{2} \times 6 = 18$ first moves, then $\binom{2}{2} \times 6 = 6$: $18 \times 6 = 108$ sequences.